# An Open Environment for the Integration of Heterogeneous Modelling Techniques and Tools

Robert Büssow[1], Wolfgang Grieskamp[1],
Winfried Heicking[2], and Stephan Herrmann[1]

[1] Technische Universität Berlin, Institut für Kommunikations- und Softwaretechnik,
FR 5–6, Franklinstr. 28/29, D–10587 Berlin. {buessow, wg, stephan}@cs.tu-berlin.de
[2] Fraunhofer-Institut für Software- und Systemtechnik ISST
Mollstr. 1, D–10178 Berlin. Winfried.Heicking@isst.fhg.de

## 1 Introduction

Even though the development of formal methods makes steady progress with respect to techniques and tool support, their acceptance in industry is still relatively low. One reason for this probably stems from the fact that most formal approaches require users to forget about their conventional techniques and tools, and to relearn in a new environment. What makes this switch even harder to accept: due to their research character, tools for formal methods can usually not compete w.r.t. convenience, stability, and degree of integration to conventional, state-of-the-art commercial tools.

Applying modern repository-based integration techniques to tools for formal methods can overcome some of these drawbacks. Moreover, an *incremental migration*, in which users can continue to apply their established conventional techniques and tools, adding formal support in degrees as desired, seems to be promising for bringing formal methods to practice.

A framework based on the idea of combining established modelling techniques and tools with formal ones has been developed in the context of the Espress project[1]: among others, Statecharts [8], a widely used graphical notation for reactive behavior, are combined with the formal notation Z [20], and with temporal logic [3, 2]. The resulting combined notation is supported by a specification methodology [7] and, last but not least, by an open and scalable tool integration environment – the design principles of which will be sketched in this paper.

*Objectives.* Our goal is to offer an environment for editing, browsing, and analyzing integrated specifications, assembled from heterogeneous formalisms – such as Statecharts, Z, temporal logic, event diagrams, etc. These formalisms are processed by different techniques, such as deduction-based analysis, model checking, systematic testing, simulation, and more. Established CASE tools (such as Statemate [9]) shall be used in combination with the strength of existing tools for

[1] The Espress project is a cooperation of industry and research institutes funded by the German Bundesministerium für Bildung, Wissenschaft, Forschung und Technologie.

formal methods (such as the deduction system Isabelle [17] or the model checker FDR [6]), and specialized tools newly developed in the application context of ESPRESS. Integration has to regard the following aspects:

- *Data Integration.* The various tools that are to be combined within the tool environment usually have their own, proprietary data format. The tool environment should provide uniform data formats, into which this data can be mapped, and should store these formats in a common repository.
- *Control Integration.* Tools may require certain data formats that are established by other tools (for example, a compiler expects a type-checker to be successfully run). The automatic control of *tool-chains* is therefore an important objective.
- *Presentation Integration.* As far as possible, interaction with the diverse tools should be standardized and follow modern design principles for GUIs. Yet, it is important to recognize that this goal cannot fully be accomplished with our approach: tools such as Statemate have their own closed world, and cannot smoothly be embedded in broader environments. It is particular crucial to deal with these "rough edges" of integration in our environment.

*Outline of the Approach.* We support a notion of a *compound heterogeneous specification document* that, in the form of a hypertext document, comprises the aggregation of models originated by different sources (cf. Sec. 2).

We define several *uniform data formats*, which may or may not be supported by each tool integrated in our framework – depending on the degree of its integration (cf. Sec. 3). Examples for formats are LaTeX, Postscript, GIF and abstract syntax.

We define interfaces of *adaptors* for tools to be integrated in our environment. A *model source adaptor* integrates an external modelling tool such as Statemate, by implementing the translation of proprietary data formats into the uniform ones. A *function adaptor* maps a set of formats into a refined set of formats, typically representing an analysis or transformation tool. The activation of both kinds of adaptors is automatically controlled by the environment. See Sec. 4.

Technically, the data formats and the adaptors for tool integration are provided in the language Java. For the definition of abstract syntax, we use the Pizza language[16], a superset of Java which provides algebraic data types and more. The code produced by the Pizza compiler can be accessed from Java programs as well. Due to its portability, powerful library support, and inter-language facilities, Java/Pizza is an ideal choice for providing interfaces to our environment. The implementation of persistency and tool control bases on the systems H-PCTE[12] and PIROL[13], which will be discussed in Sec. 5.

## 2 Compound Specification Documents

The ESPRESS methodology advocates the combination of different, specialized modelling techniques. By now, these are Statecharts, a version of dataflow-diagrams (Statemate activity charts), Z, and an extension of Z by temporal

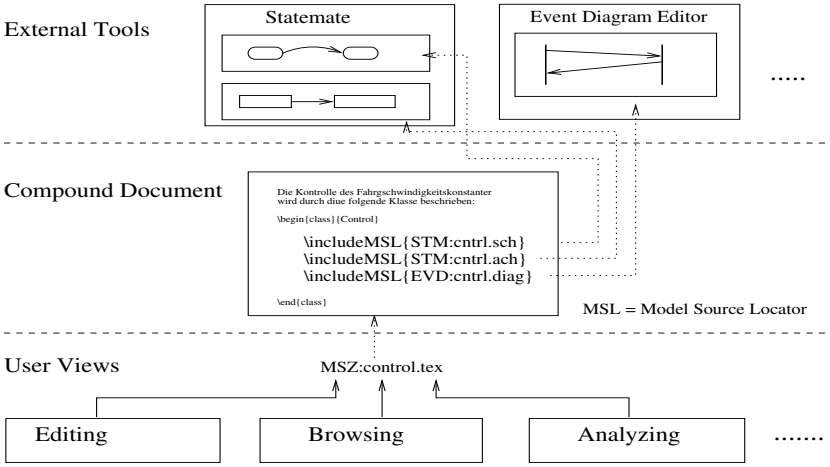



**Fig. 1.** Compound Heterogeneous Specification Documents

logic. These and possible other formalisms are combined into a Z–based notation, called $\mu\mathcal{SZ}$ [2], which is the basis for the *semantic integration* of formalisms for reactive systems that adopt the synchronous approach (such as Statecharts and temporal logic). The discussion of $\mu\mathcal{SZ}$ is out of the scope of this paper; we just mention that $\mu\mathcal{SZ}$ provides in particular means for structuring and modularisation, which allows us to cluster the different modelling techniques and provide them with uniform interfaces.

In spite of the semantical integration, different sub-models of specifications are still treated by different tools. The specification document is a *heterogeneous compound document* given in a markup-language (currently LaTeX); the references to aggregated sub-models (and also to the overall model) are described by so-called *model source locators* (MSL), a concept comparable to URLs. An MSL identifies the tool that is to provide the (sub-)model and a tool specific identifier for the desired portion. This is illustrated in Fig. 1.

The major difference between the concept of URLs and of our MSLs is that the latter are interpreted in different ways, depending on the operation to be performed on a compound document. For example, if a document is browsed, an MSL is resolved to encapsulated postscript. If it is analyzed, it is resolved to an abstract syntax term.

Fig. 2 illustrates by a screen-shot the editing of a compound document in our environment. The compound document itself is edited within the XEmacs editor, which has been extended by visualisation support for the Z notation (Z as well as arbitrary documentation may be literally entered into a compound document). MSLs referring to Statecharts and activity charts of the Statemate tool are resolved to bitmaps which are inserted into the XEmacs editing buffer. When the user changes the charts in the Statemate tool, the bitmaps under XEmacs

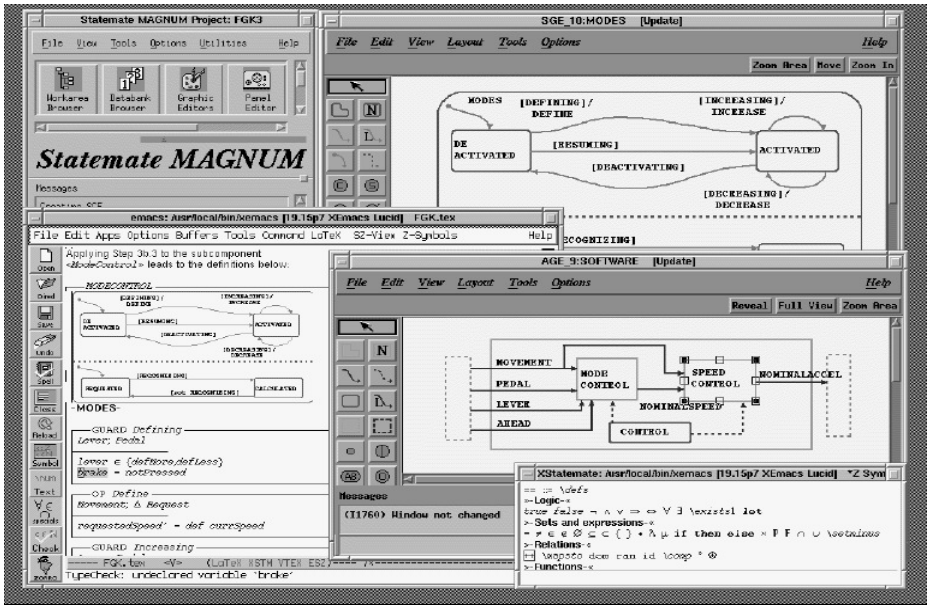



**Fig. 2.** Editing a Compound Document with Statemate and XEmacs

will be updated automatically. In the sequel, we will mainly concentrate on the technical concepts which make editing, browsing and analyzing of compound documents possible.

## 3 Data Integration

In our environment a collection of tools, textual and graphical editors, simulators, analysis tools, code generators, etc., work together on an heterogeneous compound specification document. To make this possible, a uniform data representation is indispensable.

To this end, we define a set of standardized *data formats*. Each MSL is dynamically associated with a set of such formats, which represent different views of the contents of the MSL. Establishing a particular format for an MSL may require invoking tool adaptors as discussed in the next section, and may generally fail. For example, establishing the format of type-checked abstract syntax requires (at least) to run a type checker, which may reject establishing this format because of type-errors.

Formats are attributed with a *qualifier*, which describe the role the format plays for the specification (such as "the test-cases derived from this specification"). This is illustrated by the tool chain in Fig. 3, as explained in the next section.

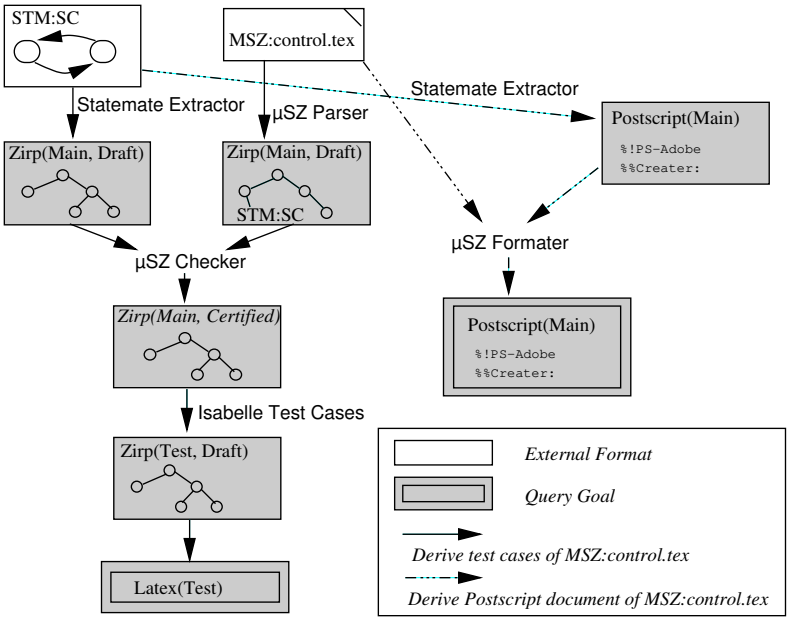



**Fig. 3.** Tool Chains: Driving of Format Queries

The most elaborated data format we provide is abstract syntax (called ZIRP, the Z Intermediate RePresentation format). This format is a compact representation of the full Z language, together with the features of our integration formalism, $\mu\mathcal{SZ}$. In addition to a qualifier, a ZIRP format is parameterized by a *state*. States signify e.g. whether the abstract syntax is type-checked ("certified"), and may imply that the abstract syntax is annotated with additional information, such as e.g. type information.

In general, it is possible for a tool developer to add new data formats to the environment, or to refine existing ones (in particular, to refine ZIRP). For example, if an extended type checker for Z is added to the environment, it may define new kinds of type-annotations, which are asserted by a state, say, "extended-certified". These new annotations may then be used by other new tools, whereas existing tools don't need to see them. As another example, a proof-checker may define a totally private format "proof-script", which, associated with an MSL, may be used to replay proofs on a specification with this particular checker.

## 4 Control Integration

For an integration that exceeds the plain data format level, we define uniform interfaces of *adaptors* for tools. The notion of an adaptor illustrates that some

functionality is plugged into the environment, no matter if the adaptor itself implements it, or if it just realizes a wrapper for another tool. We distinguish the following kinds of adaptors:

- *Model source adaptors* connect external modelling tools and techniques. They translate tool specific data into a known format and, by that, create initial formats from MSLs. They also keep track of whether the tool specific data is outdated, and formats derived by it need to be invalidated.
- *Function adaptors* implement forms of analysis or transformations and, by that, transform formats and/or derive new formats from existing ones.

On registration, a model source adaptor tells the environment which tool-identifier (part of on MSL, such as `STM`) it serves, and which format types it can provide. A function adaptor is registered with the set of required input formats, and the output formats it will produce on success. This information enables a *data-driven* control integration. The basic scenario is that a user (or tool) requests a certain data format for an MSL from the environment; necessary actions to establish this format are then automatically performed by the environment.

Fig. 3 illustrates the process to resolve such requests. The formats ZIRP, Postscript and LaTeX are involved. The format qualifier *Main* addresses the main specification document, the qualifier *Test* the test-cases derived for this document. The state *Draft* is associated with unchecked, the state *Certified* with type-checked abstract syntax.

Given a compound document identified by the MSL `MSZ:control.tex` with a behavioural sub-model identified by `STM:SC`, two requests are illustrated: get *Latex*(*Test*) of `MSZ:control.tex`, that is a LaTeX-representation of the derived test cases; and get *Postcript*(*Main*) of `MSZ:control.tex`, that is a postscript version of the overall specification.

Two model source adaptors, the Statemate extractor and the $\mu\mathcal{SZ}$ parser for compound documents, are involved. The Statemate extractor translates statecharts into the ZIRP format or into postscript. The $\mu\mathcal{SZ}$ type-checker is integrated by a function adaptor which merges the abstract syntax resulting from the statechart and the compound document, and certifies it. Test cases are calculated on the basis of a function adaptor which connects to the theorem prover Isabelle.

The example of the Isabelle test case adaptor illustrates some further aspects: First, this function adaptor will internally convert the ZIRP syntax into the HOL-Z encoding of Z in Isabelle [14], invoke an appropriate operation under HOL-Z, and convert the result back to ZIRP. Thus, hidden inside of a function adaptor, a mapping between uniform and proprietary data-formats may take place. Secondly, the generation of test-cases is of course not the only function provided by a powerful tool such as Isabelle. In fact, an adaptor does not necessarily coincide with a physical tool instance: there may be several adaptors connecting to the same tool instance. Each physical tool in turn needs to be started only once for each user session and runs as a background process waiting for requests from the environment. This takes into account that tools like Isabelle have a considerable start-up time.

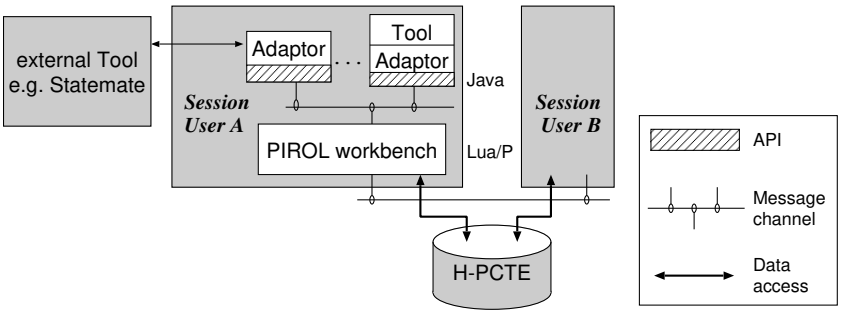



**Fig. 4.** Architecture of PIROL

Note that adaptors are generally considered to operate non-interactively. Analysis operations such as type-checking, which produce diagnostics, store the messages as a particular format of an MSL, which can be later on retrieved by browsing tools. Yet, some adaptors may require configuration parameters; the interactive definition of these parameters is generically realized by our environment.

By giving control over invocation of adaptors to the environment, editors and browsers are completely decoupled from any computations. Adaptors can be plugged in and out without modifying the visual tools. Even the algorithm for animation of the specification can be implemented without knowledge of any GUI and the control integration of the environment may trigger existing display tools to continuously display the animation state.

## 5 Implementation

The Espress tool environment is implemented by a multi–layer architecture. Fig. 4 depicts the main components: the H-PCTE persistent object storage, the PIROL workbench, and a Java-based application program interface (API) for formats and adaptors.

*Persistent object storage.* Basic integration of persistent data is managed by a central object management system (OMS). We employ H-PCTE[12] which implements a structurally object oriented database according to the ISO standard PCTE[18]. PCTE is especially designed as framework for software engineering environments providing services that exceed a plain database system.

*PIROL Workbench.* Control integration is handled by the PIROL workbench[13], which at the current state runs as a background-process for each user session. The workbench provides (possibly cached) access to the OMS and executes methods on persistent objects. This lifts persistent objects to full object orientation. OMS and workbench together form the full fledged active repository, that hosts

all build algorithms for the driving of tool chains. These algorithms are implemented by means of the object oriented repository language Lua/P[2].

Formats, in particular abstract syntax trees, are treated as atomic repository objects which are opaque for the PIROL workbench. The desire to define an abstract syntax by the repository object model would yield one repository object from each node of the abstract syntax – which would be unbearable due to the overhead each repository object causes in terms of administrative data and access execution time. Even medium sized specifications easily produce syntax trees with several thousands of nodes. For this reason a second level of data modelling is deferred to the next architectural level, the Java-based tool API.

Tools technically converse only with one component: the PIROL workbench. This is the key to a scalable environment in which tools can be plugged in and out.

*Java-based API.* The definition of the format data types, the interfaces for tool adaptors, and general access to the repository is provided by an API given in Java (resp. Pizza for abstract data types). This API provides a complete abstraction and specialization for the concerns of Espress of the underlying repository architecture. Repository objects are thereby interfaced by according Java proxies objects. The message-based coupling of Java with the PIROL workbench is done by means of Java's JNI (Java Native Interface).

The JNI also supports the implementation of adaptors for tools which provide a C interface, such as Statemate and its so-called dataport. Since Java provides a powerful general programming library including reflection, other forms of adaptors which base on textual exchange formats are easily implemented as well. For example, the Isabelle adaptor uses a generic parser/unparser for mapping ZIRP to SML Terms, and is implemented by reflection over the ZIRP format type. Reflection capabilities have also been used to construct a *binding compiler* which produces strongly-typed interfaces to Java classes for the functional language OPAL; this way, the adaptor for the $\mu\mathcal{SZ}$ type-checker ESZ, written in OPAL, has been realized. Moreover, an IDL mapping for Java and a Java ORB are available, such that Java-based adaptor implementations can connect to the CORBA world. Last but not least, adaptors and other tools can be written from scratch in Java resp. Pizza itself – this is supported by Java and Pizza libraries tailored for manipulating formats.

## 6 Related Work

Few projects approach the task of integrating tools for formal methods and existing commercial CASE tools in all aspects discussed in this paper. The approach of the UniForM Workbench[15] is quite similar to our's. Wheras our integration languages Java and Lua/P are object oriented, UniForM employs a functional language (Haskell) for this task. Also the architecture differs: UniForM shows

[2] Lua/P is an instance of the Lua[11] language family specially adapted for the PIROL repository

a greater number of general components, called managers. We implement the functionality of these managers by just one component, the workbench, thus reducing the number of interfaces. A closer comparison of these two system should trigger further interesting evolution.

Other approaches like the Concurrency Factory[4] or AutoFocus[10] present more homogenous environments, with most tools written from scratch and less emphasis on an active repository as a component for integration of persistent data and control.

## 7 Conclusion

We have presented the design principles of a tool environment which can integrate heterogeneous formal and non-formal modelling as well as analysis and transformation techniques. Our approach bases on the discernement that we have to reuse and integrate existing tools to bring formal methods to practice. We emphasise that the objective of a tool environment for formal methods has to take advantage of existing, sophisticated tools rather then reinventing them – which would be intractable in praxis nowadays.

Our approach anticipates that we have to live with "rough edges" if we want to integrate tools for the application of formal methods in an environment which has a chance for acceptance in industrial praxis. A major principle of our work is to grind these edges, instead of trying to eliminate them. In particular, we give solutions on how external modelling techniques can be integrated in our framework – let them be just non-integrated text-editors such as "vi" or comprehensive CASE tools such as Statemate.

Dispite of all constraints given by external tools a high level of integration has been reached. This contributes to the convenience of tool support for formal methods. By using an open repository–based environment as integration platform, separation of concerns is enforced and interdependencies between tools are reduced. Tools request services only from one component (the workbench) but indirectly make use of the functionality of the whole environment. This separation of concerns helps to add new features to the environment without disturbing existing functionality. *Version- and access control* are tasks of the repository proper. Universal tools for handling versions and configuring access control easily cooperate with all other tools in the environment. The same holds for general repository browsers, printing utilities etc.

Granularity of data modelling is a crucial issue for tight data integration. The approach of ISO PCTE to merge concepts for fine grained objects into PCTE [19] must be considered unsuccessfull because of the large difficulty to implement the standard. Our approach draws a clear line between the medium grained repository object model (given in Lua/P) and the very fine grained abstract syntax (given in Pizza). The first level grants the workbench access to all information needed by environment services. The second level provides the basis for close cooperation between tools, yet pertaining a sufficient performance of the system.

We have shown how the integration of data, control and presentation is implemented. In addition to these aspects the ECMA reference model[5] mentions "process integration" and "framework integration". Both blend well with the concepts introduced so far. The PIROL repository object model is designed also for support of the *development process* in terms of organisation and methodology. The term *framework integration* captures services for common administrative tasks like tool configuration and administration of users and projects. Tool configuration is handled by the tool libraries in conjunction with the workbench. All other administration is orthogonal to any other service and support can be added easily.

The environment represented in this paper has been implemented and applied in the course of the ESPRESS project. Currently, we have integrated the foreign tools Statemate, XEmacs, Isabelle, FDR, and the ESPRESS specific tools ESZ (a Z/$\mu\mathcal{SZ}$ type checker) and ZAP (a compiler for Z). The applicability of our approach has been demonstrated in two case-studies with industrial background. One case-study is the specification of a traffic light system [1], the second [7] is the specification of an intelligent cruise control.

## References


1. R. Büssow, R. Geisler, and M. Klar. Specifying safety-critical embedded systems with statecharts and Z: A case study. Accepted for publication at ETAPS'98, 1998.
2. Robert Büssow, Robert Geisler, Wolfgang Grieskamp, and Marcus Klar. The $\mu\mathcal{SZ}$ notation version 1.0. Technical Report 97-26, Technische Universität Berlin, Fachbereich Informatik, December 1997.
3. Robert Büssow and Wolfgang Grieskamp. Combining Z and temporal interval logics for the formalization of properties and behaviors of embedded systems. In R. K. Shyamasundar and K. Ueda, editors, *Advances in Computing Science – Asian '97*, volume 1345 of *LNCS*, pages 46–56. Springer-Verlag, 1997.
4. R. Cleaveland, J.N. Gada, P.M. Lewis, S.A. Smolka, O. Sokolsky, and S. Zhang. The concurrency factory — practical tools for specification, simulation, verification, and implementation of concurrent systems. In *Proceedings of the DIMACS Workshop on Specification of Parallel Algorithms*, Princeton, NJ, May 1994.
5. Reference Model for Frameworks of Software Engineering Environments – ECMA TR/55 3rd edition. Technical report, European Computer Manufacturers Association (ECMA), June 1993.
6. Formal Systems (Europe) Ltd. *Failures Divergence Refinement, FDR2 User Manual*, 1997.
7. Wolfgang Grieskamp, Maritta Heisel, and Heiko Dörr. Specifying safety-critical embedded systems with statecharts and Z: An agenda for cyclic software components. Accepted for publication at ETAPS'98, 1998.
8. David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, June 1987.
9. David Harel, Hagi Lachover, Amnon Naamad, Amir Pnueli, Michal Politi, Rivi Sherman, Aharon Shtull-Trauring, and Mark Trakhtenbrot. Statemate: A working environment for the development of complex reactive systems. *IEEE Transactions on Software Engineering*, 16 No. 4, April 1990.

10. F. Huber, B. Schätz, A. Schmidt, and K. Spies. AutoFocus - a tool for distributed systems specification. In *FTRTFT'96 – Formal Techniques in Real-Time and Fault-Tolerant Systems*, volume 1135 of *LNCS*, pages pp. 467–470. Springer Verlag, 1996.
11. R. Ierusalimschy, L. H. de Figueiredo, and W. Celes. Lua—an extensible extension language. *Software: Practice and Experience*, 26(6):635–652, 1996.
12. Udo Kelter. H–PCTE — a high–performance object management system for system development environments. In *Proc. COMPSAC '92*, pages 45–50, Chicago, Illinois, September 1992.
13. Boris Groth; Stephan Herrmann; Stefan Jähnichen; Wilfried Koch. Project Integrating Reference Object Library (PIROL): An object–oriented mutliple–view SEE. In *Proceedings of the 7th Conference on Software Engineering Environments (SEE'95)*, Noordwijkerhout, Holland, April 1995.
14. Kolyang, T. Santen, and B. Wolff. A structure preserving encoding of Z in Isabelle/HOL. In J. von Wright, J. Grundy, and J. Harrison, editors, *Theorem Proving in Higher-Order Logics*, LNCS 1125, pages 283–298. Springer-Verlag, 1996.
15. B. Krieg-Brückner, J. Peleska, E.-R. Olderog, D. Balzer, and A. Baer. Universelle Entwicklungsumgebung für Formale Methoden (UniForM Workbench). Informatik Bericht 8/95, Universität Bremen, 1995.
16. Martin Odersky and Philip Wadler. Pizza into Java: Translating theory into practice. In *Proc. 24th ACM Symposium on Principles of Programming Languages*, January 1997.
17. L. C. Paulson. *Isabelle – A Generic Theorem Prover.* LNCS 828. Springer-Verlag, 1994.
18. ISO/IEC 13719-1: Portable Common Tool Environment (PCTE). Abstract specification, International Organization for Standardization (ISO), 1995.
19. Amendment 1 to ISO/IEC 13719-1: Fine-grain object extensions. Technical report, International Organization for Standardization (ISO), 1995.
20. J. M. Spivey. *The Z Notation: A Reference Manual.* Prentice Hall International Series in Computer Science, 2nd edition, 1992.